

数字系统设计实验报告

无 28 周璟昊

实验一、1*N 乘累加树阵列

乘累加树主要由 MAC_NUM 与 MAC_KUM 两参数决定，其中 MAC_NUM 对应 1*N 阵列中的 N，也即 spatial_for N 的分解；而 MAC_KUM 参数则决定每棵树由多少个乘法单元组成，是 spatial_for K 的分解。

需要注意的是，为了保证 logN 结构的乘累加树实现，乘法单元 MAC_KUM 数量需要为 2^N 形式，故采用 16；此时由于输入矩阵宽不能整除 16，需要由补零操作实现矩阵乘法。

前仿结果如下图所示，成功通过乘累加树阵列实现了矩阵乘的运算；从图中可以看出，1*N 的乘累加树阵列，一次会输出 N 个结果（MAC_NUM=10）。

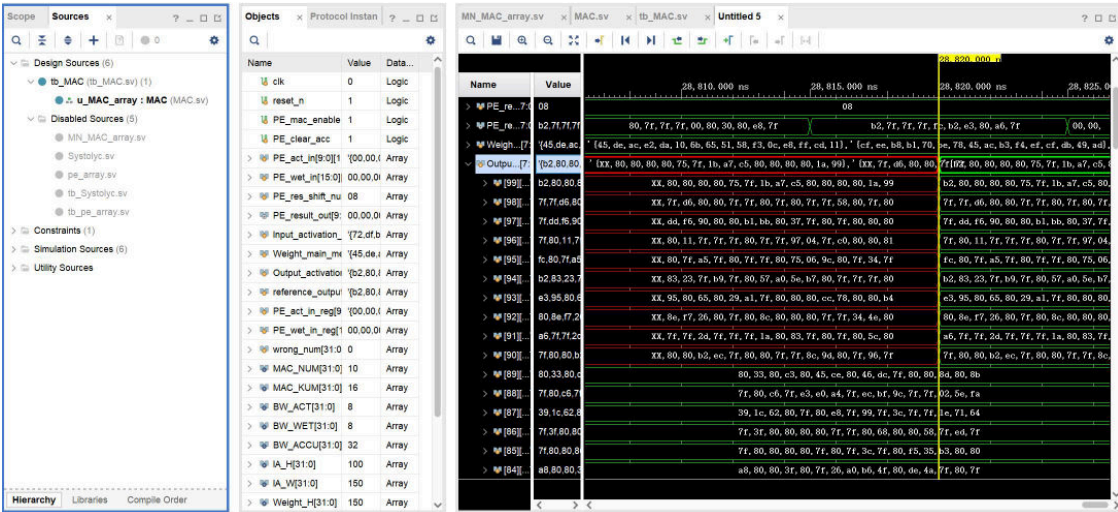


图 1. 1*N 乘累加树阵列前仿结果

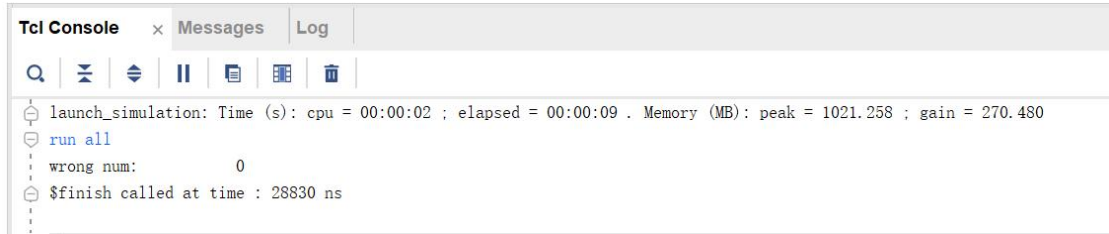


图 2. 1*N 阵列仿真耗时

可见耗时为 28830ns，然而实际时间应该可以更短，由于开始设计时错误将乘法、加法单元理解为中间寄存器，导致一次输入需要有数拍才能算出结果，导致耗费了更多时钟周期，而没有去提高时钟频率；但寄存器的存在也使得一次计算可以以流水线形式输入。

如下为综合优化结果，可见其中有关 IO 端口的优化情况异常，考虑为调试时，将过多

Summary

Resource	Utilization	Available	Utilization %
LUT	20217	20800	97.20
FF	9172	41600	22.05
IO	13467	150	8978.00

图 3. 1*N 阵列 Utilization

的寄存器输出到 testbench 文件中进行 debug，导致器件的 IO 情况较坏。而在时序分析方面，WNS 仍然有一定冗余，可以进一步提高时钟频率；此外，使用寄存器分隔形势下也是逐次计算输入，可以尝试进一步优化使其各次输入之间也以流水线的形式输入进行优化。

Design Timing Summary			
Setup		Hold	Pulse Width
Worst Negative Slack (WNS): 2.827 ns		Worst Hold Slack (WHS): 0.129 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns		Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0		Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 7760		Total Number of Endpoints: 7760	Total Number of Endpoints: 9173
All user specified timing constraints are met.			

图 4. 1*N 阵列的时序情况

算力: $100M \times 16 \times 10 = 16Gops$
最高时钟频率: $1 / (10 - 2.827) \text{ ns} = 139.41MHz$
计算量/算力 = $100 \times 150 \times 16 / 16Gops = 15us$ ，而仿真用时 28.83us，可见阵列利用率为 52.03%
IA 带宽需求: $8b \times 10 \times 16 \times 100M = 128Gbps$
W 带宽需求: $8b \times 16 \times 100M = 12.8Gbps$
OA 带宽需求: $8b \times 10 \times 100M = 8Gbps$
IA 片上 buffer 容量: $8b \times 10 \times 16 = 1280bit$
W 片上 buffer 容量: $8b \times 60 = 128bit$
OA 片上 buffer 容量: $8b \times 10 = 80bit$
IA 片外访问总 bit 数 (包含补 0): $16 \times 100 \times 160 \times 8bit$
OA 片外写回总 bit 数: $16 \times 100 \times 8bit$
W 片外访问总 bit 数 (包含补 0): $16 \times 160 \times 10 \times 8bit$

实验二、1*N 脉动阵列

在 1*N 脉动阵列设计中，我理解其为如下图所示阵列，故其同样可以有两个重要参数，一个是输入脉动长度 MAC_KUM，一个是阵列长度 MAC_NUM (也即 N)；二者分别代表了脉动输入的空间分化 (spatial_for k) 与并行运算情况 (spatial_for N)。

然而在实际尝试过程中发现，如果 MAC_KUM 设定为与 MAC_NUM 不同的参数，需要考虑二者大小问题，引入较为复杂的 output_reg 更新机制，导致仿真极为缓慢，故最后设计时选取 MAC_KUM=MAC_NUM=10 的参数。

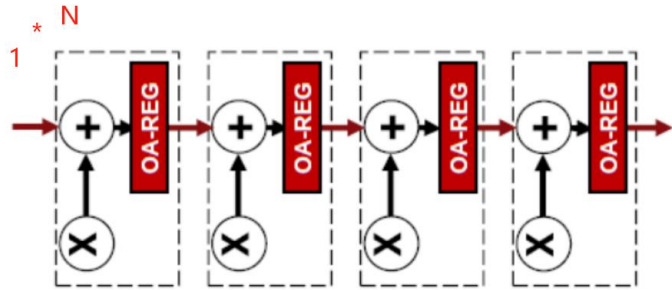


图 5. 1*N 脉动阵列理解示意图

仿真结果如下，可以发现其耗时显著较 1*N 乘累加树阵列长，考虑是由于脉动阵列在 1 维情况下计算效果并不显著突出导致，其此时脉动转播并不能像乘累加树一样以 logN 级别进行并行的流水计算。1*N 脉动阵列同样一次写入 N 个结果。

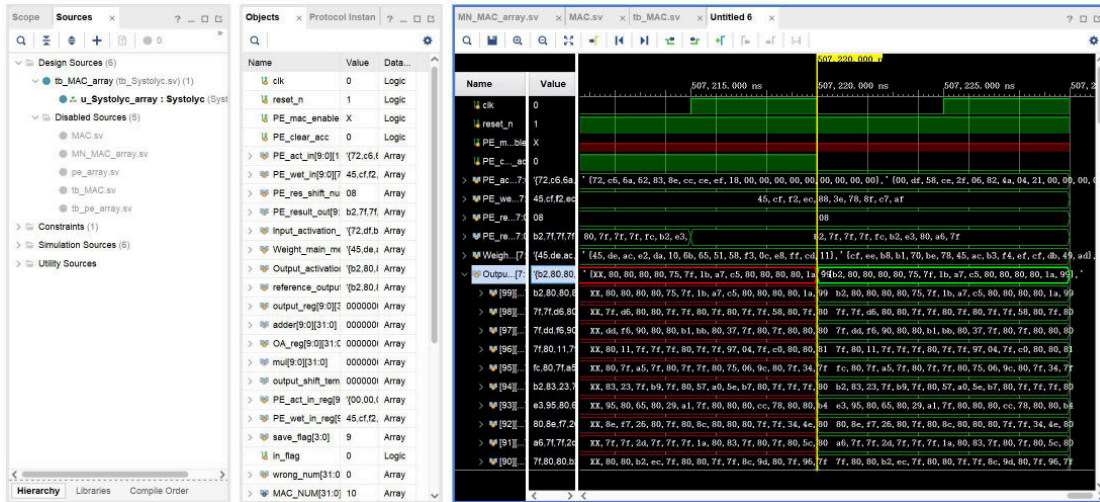


图 6. 1*N 脉动阵列前仿结果

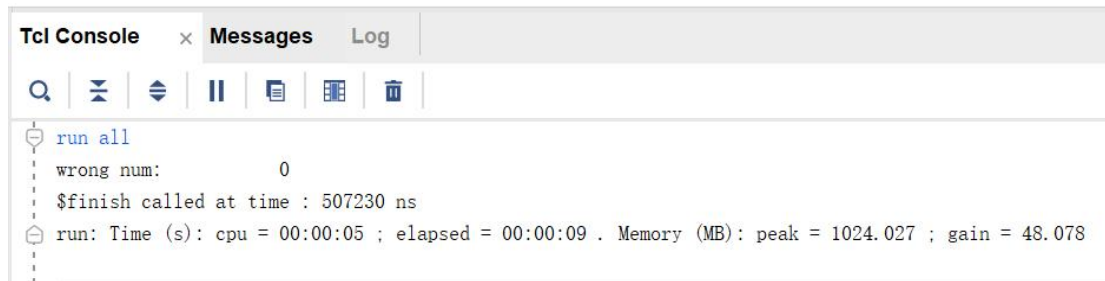


图 7. 脉动阵列仿真耗时

如下为优化结果，可以看到在移除了 debug 时的输出管教后，IO 异常不那么突出，余下可能为仿真板型号所导致的错误；结合图 9 时序分析情况来看，100MHz 的时钟频率下 WNS 仍为正，可以进一步缩短时钟周期。

Summary				
Resource	Utilization	Available	Utilization %	
LUT		3402	20800	16.36
FF		2935	41600	7.06
IO		4896	150	3264.00

图 8. 脉动阵列 utilization

Design Timing Summary		
Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 2.610 ns	Worst Hold Slack (WHS): 0.132 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 2365	Total Number of Endpoints: 2365	Total Number of Endpoints: 2936
All user specified timing constraints are met.		

图 9. 脉动阵列时序情况

算力: $100 \times 10 = 1\text{Gops}$

最高时钟频率: $1 / (10 - 2.610) \text{ ns} = 135.32\text{MHz}$

计算量/算力 = $100 \times 150 \times 16 / 1\text{Gops} = 240\text{us}$ ，而仿真用时 507.23us，可见阵列利用率为 47.32%

IA 带宽需求: $8b \times 20 \times 100M = 1600Mbps$
W 带宽需求: $8b \times 10 \times 100M = 800Mbps$
OA 带宽需求: $8b \times 10 \times 100M = 8Gbps$
IA 片上 buffer 容量: $8b \times 10 \times 20 = 1600bit$
W 片上 buffer 容量: $8b \times 10 = 80bit$
OA 片上 buffer 容量: $8b \times 10 = 80bit$
IA 片外访问总 bit 数 (包含补 0): $16 \times 100 \times 150 \times 8bit$
OA 片外写回总 bit 数: $16 \times 100 \times 8bit$
W 片外访问总 bit 数 (包含补 0): $16 \times 150 \times 10 \times 8bit$

根据仿真资源来看, 有如下解释与问题, 1、阵列利用率; 2、IA 贷款需求: 考虑到一维脉动阵列不能实现乘累加树的流水运算, 以及为了方便设计, 一次性将 $2 \times MAC_NUM$ 个数(含有补 0 值) 导入到片上寄存器中流水执行, 而每执行 $2 \times MAC_NUM$ 后才再次从片外读取 $2 \times MAC_NUM$ 个数据。故考虑如上因素, 认为其带宽需求、阵列利用率与计算速率效果较差是正常的情况。

实验三、M*N 乘累加树阵列

取 MAC_MUM 为 4 (即参数 M), 以便与输出矩阵宽度匹配, 再实例化 $16/4=4$ 个实验一中的 $1 \times N$ 阵列, 即可得到 $M \times N$ 的乘累加树阵列, 仿真结果如下图所示, 可见 $M \times N$ 乘累加树阵列一次写回 $MAC_MUM \times MAC_NUM$ (即 $M \times N$) 个数。仿真耗时上来看, 也几乎相当于 $1 \times N$ 阵列总时长除以 $MAC_MUM(4)$, 整体与预期相符。

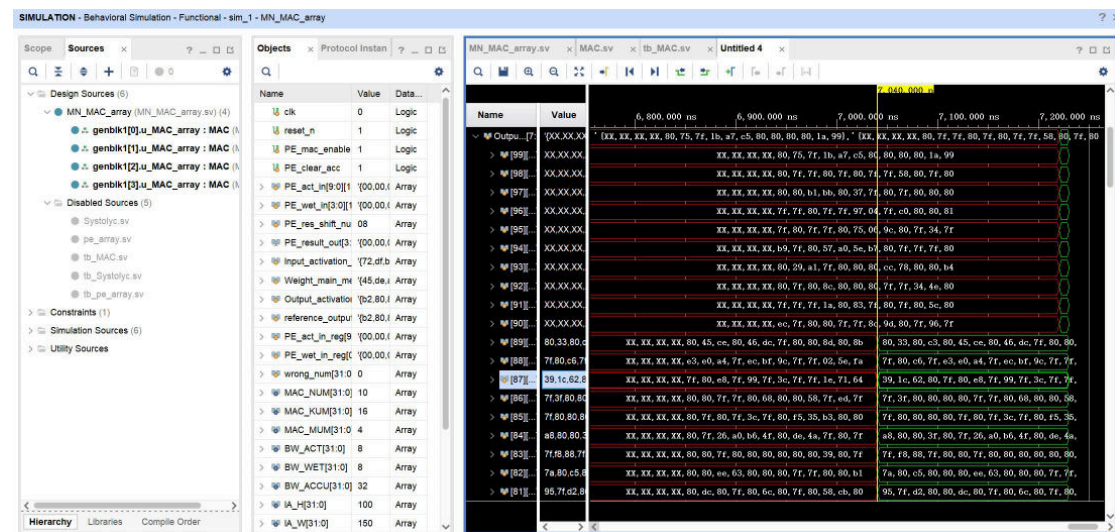


图 10. $M \times N$ 乘累加树阵列仿真结果

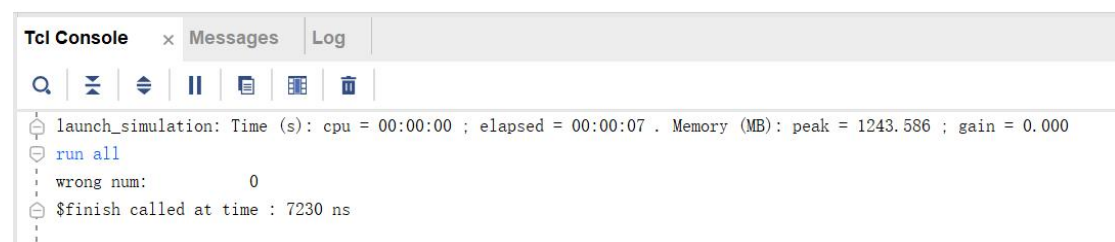


图 11. $M \times N$ 阵列仿真耗时

其综合实现的 utilization 与时序情况如下图 12、13 所示，可见其时序情况与 1*N 阵列相同，而在此基础上有了 4 倍（MAC_MUM）的算力/输入输出带宽，基本与 1*N 阵列相同，故此处略去数据，只需要将 1*N 部分数据进行乘或除以 4 的操作即可。

Summary			
Resource	Utilization	Available	Utilization %
LUT	19940	20800	95.87
FF	9172	41600	22.05
IO	2907	150	1938.00

图 12. M*N 阵列 utilization

Design Timing Summary			
Setup	Hold	Pulse Width	
Worst Negative Slack (WNS): 2.827 ns	Worst Hold Slack (WHS): 0.129 ns	Worst Pulse Width Slack (WPWS): 4.500 ns	
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns	
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	
Total Number of Endpoints: 7760	Total Number of Endpoints: 7760	Total Number of Endpoints: 9173	
All user specified timing constraints are met.			

图 13. M*N 阵列时序情况

但考虑 M*N 的脉动阵列，其可保持脉动与流水形式输入，故其应该相较于 1*N 的脉动阵列会有较为明显的提升，而不止是像乘累加阵列的并排效果。

思考题

1、非 GEMM 方式计算卷积中，如 Winograd 卷积，该如何控制数据向阵列的传输？

需要按照 Winograd 卷积的分块规则，将对应矩阵区域按块进行读取、缓存，同时尽可能提高复用率

2、在你之前设计的基础上，如果有更大的片上 buffer (或者有一个新的 shared buffer)，你会怎么重新进行 mapping 的设计？会有什么样的好处？（提示：40nm 工艺下，外部访存的功耗是内部访存的 3~6 倍以上）

我会从数据缓存与映射策略两方面考虑进行修改。在数据缓存方面，更大的 buffer 允许更多的读入/写出数据缓存，可以进一步减少对片外存储的访问次数，减少时间与能量开销；在映射策略方面，在 Weight Stationary 或 Input Stationary 等情况下，更大的 buffer 均可以允许一次导入更多的数值，同样减少对片外存储的访问。

3、在你之前设计的基础上，如果实际不能满足你的带宽需求，比如参考代码的 0A 带宽需求是 8000Mbps，但实际只能做到 800Mbps，那该如何重新进行设计？可以进行一个 case study。

可以从数据角度、设计角度进行处理。在数据方面，可以进一步对数据进行压缩和量化（精度允许的范围内），减小输入输出的带宽；在设计角度，则可以进行一个数据的预处理预加载，或者增添一个数据读取的时钟周期，以匹配传输带宽。

4、在你之前设计的基础上，如果阵列的尺寸并不是新任务中输入矩阵尺寸的因数，例如在参考代码中，阵列尺寸是 1×10 ，来了个新任务输入激活值矩阵尺寸是 102×128 ，权重矩阵尺寸是 128×18 。那么你该如何进行 mapping？或者如何进行更好的阵列设计？

如果阵列与任务尺寸不匹配，可以进行如下处理：1、对于如 FPGA 类硬件，只需要修改参数即可调整阵列排布，实现 mapping；2、对于非 FPGA 类硬件，则可以通过分块处理、不匹配处补 0 的操作，以实现矩阵的运算，但此时使用率降低。

为了进行更好的阵列设计，应该选取 M、N 更为接近、适中的数值，比如 1×10 变更为 2×5 等，以尽可能提高阵列的使用率。

5、在你之前设计的基础上，试着去搜索一下更好的 mapping 方案。（延时更低、带宽需求更低、访问 bit 数更低等）

数据复用：利用行缓冲存储输入特征图行，供多个卷积窗口复用，减少带宽。

流水线设计：数据传输与计算重叠，如前一窗口计算时，后一窗口数据预取，降低延时。

6、试着优化你的设计，使其能够跑到更高的时钟/更短的任务时间。

- 提高时钟频率；
- 提高并行率，如一维脉动阵列转换为二维脉动阵列、优化乘累加树的各行列输入间流水线输入；
- 匹配输入输出带宽与加法、乘法单元的计算频率，以达到最优匹配的阵列结构。